

Práctica de Búsqueda Local

Aleix Ortigosa

David Castro

Universidad Politécnica de Cataluña



30 de marzo de 2025

Índice

1. Introducción al problema	2
2. Representación del estado	3
2.1. Las clases Estado y Graph	4
2.2. Soluciones iniciales	6
2.2.1. Solución simple	6
2.2.2. Solución voraz	7
2.3. Operadores	7
2.3.1. Operador de cambio de nodo destino	7
2.3.2. Operador de intercambio de nodos destino	8
2.4. Heurística	8
3. Experimentación	9
3.1. Elección sobre el conjunto de operadores	9
3.2. Elección de la solución inicial	12
3.3. Elección de parámetros para Simulated Annealing	14
3.4. Análisis del tiempo de ejecución	21
3.5. Proporción centros de datos/sensores que acaban en centros vacíos	22
3.6. Afectación al coste al añadir centros	24
3.7. Ajuste de la función heurística para otro escenario	26
4. Trabajo de Innovación	29
4.1. Tema	29
4.2. Reparto del trabajo	30
4.3. Referencias	30
4.4. Dificultades	31

1. Introducción al problema

El escenario del problema planteado es una red de sensores que capturan y emiten datos y centros que los pueden recibir. Los sensores tienen cierta capacidad de almacenamiento que podrán transmitir junto al resto de datos que les lleguen. Queremos encontrar una configuración de conexiones entre estos sensores y centros para que los centros reciban el máximo volumen de datos con el mínimo coste posible.

El escenario está presentado en un área geográfica de 100×100 km². Tanto los sensores como los centros de datos tienen coordenadas x e y en el plano, esto hace que tengamos ciertas distancias entre todo par de elementos, que estarán dadas por la fórmula:

$$d(x, y) = \sqrt{(x_i - y_i)^2 + (x_j - y_j)^2}$$

Según esta distancia y el volumen que se intenta transmitir por una conexión, obtenemos el coste ya mencionado y que nos es dado por esta fórmula:

$$\text{coste}(x, y) = d(x, y)^2 \times v(x)$$

El coste total que se busca minimizar será la suma del coste de todas las conexiones que se hayan formado. Aparte de estas propiedades de los elementos del problema existen dos restricciones que se describen con esta regla:

Un elemento puede recibir n volumen de datos desde m nodos distintos.

Para sensores, siendo c su capacidad por defecto:

$$c \in \{1MB, 2MB, 5MB\}, \quad n = 2 * c, \quad m = 3$$

Para centros:

$$n = 150MB, \quad m = 25$$

Además hay que tener en cuenta que cada sensor puede formar ninguna o una sola conexión con otro sensor o un centro, y un centro no se conectará a ningún otro elemento, solo permite que se conecten a él. Las conexiones que admite un centro son 25 y para un sensor son 3.

Para que una configuración de la red sea solución tiene que cumplir los siguientes criterios:

1. La red ha de conectar a todos los sensores a un centro directa o indirectamente.
2. Hay una proporción equilibrada entre el volumen transmitido que tenderá al máximo y el coste total que tenderá al mínimo.

Nos encontramos con un problema que cae en la clásica categoría de optimización de red. Es un problema de búsqueda local, ya que cumple con estas características necesarias:

1. Espacio de soluciones bien definido : Una solución se define por la asignación de conexiones entre sensores y centros de datos, formando una red.
2. Función de evaluación posible : se puede definir una función objetivo que mida la calidad de una solución, en este caso haciendo una ponderación entre el coste y el volumen de datos recibido por los centros. Con el objetivo de minimizar el coste y maximizar el volumen de datos.
3. Vecindario de soluciones : Se pueden generar soluciones vecinas modificando la red de conexiones.
4. Falta una solución óptima global : A primera vista, no existe un algoritmo exacto eficiente que garantice encontrar la solución óptima en tiempo razonable, ya que el problema involucra muchas combinaciones posibles de conexiones, en cuanto el número de elementos no sea mínimo.
5. Posibilidad de mejorar soluciones iterativamente : Se puede mejorar una solución inicial aplicando técnicas como Hill Climbing o Simulated Annealing usando el vecindario de soluciones.

2. Representación del estado

Como se aconseja en el enunciado de la práctica, hemos modelado la configuración de sensores y centros que conforman un estado cualquiera como un grafo dirigido acíclico o DAG, donde estos son los nodos y toda arista entre ellos es una conexión. Este modelaje del problema consigue que podamos ver de forma intuitiva los operadores a aplicar y las estructuras utilizadas en la implementación. Además de cada nodo del grafo solo sale 1 arista si es un sensor y 0 aristas si es un centro.

Lo hemos modelado como un DAG, por estos motivos:

1. las direcciones de las conexiones son unidireccionales
2. No pueden existir ciclos teniendo en cuenta las restricciones de que de cada sensor solo sale 1 arista y que todos los sensores tienen que estar conectados de forma directa o indirecta a un centro de datos.
3. Tener una lista de adjacencias facilita la comprobación de las restricciones y criterios de solución
4. Eficiencia de la estructura para los cálculos de coste y volumen.
5. Generación fácil de vecinos modificando la lista de adyacencias.
6. Uso y conocimiento sobre los DAG.

Sea S el número de sensores y C el número de centros y solo fijándonos en la estructura de datos y no en las restricciones, cada sensor puede conectarse a solo 1 otro sensor o centro, y teniendo en cuenta que lo puede hacer cada sensor la cota inicial del espacio de búsqueda es de

$$O(S + C - 1)^S$$

Las restricciones como que el grado de entrada de los sensores sea menor o igual a 3 y que para los centros sea menor o igual a 25 reducen notablemente esa cota pero sigue siendo intratable.

2.1. Las clases Estado y Graph

Funcionalmente estas dos clases serían la misma, el único atributo de Estado es un objeto Graph y sus funciones son one-liners con llamadas a su respectiva función de Graph.

La razón de esto simplemente es que decidimos distinguir explícitamente las 2 capas que forman la representación de un estado. Por un lado, sabemos que para representar el DAG tenemos que usar ciertas estructuras y que para aplicar los operadores hay que recorrer el grafo y realizar ciertas acciones... pero conceptualmente, el estado no debe pensar en eso, solo se preocupa de las acciones a alto nivel, aplica el operador y delega, no necesita saber los cambios internos que se tendrán que realizar.

La clase Graph contiene la representación del DAG como ya hemos dicho, la implementación de los operadores, las soluciones iniciales, la heurística y demás funciones para facilitar información a los algoritmos de búsqueda.

Explicaremos sus atributos y sus funciones más importantes, pero antes hay que hablar de una clase que utiliza, la clase Edge.

Esta es simplemente la representación de una arista, pero para nuestro problema tendrá ciertos atributos aparte de los clásicos origen y destino:

- **distancia:** El cálculo del coste se hace sobre las conexiones, estas aristas representan cada conexión y por tanto guardamos en ellas la distancia de sus nodos u-v.
- **volumenReal:** Volumen de datos que transmite y llega a su destino.
- **volumenFalso:** volumen hipotético, es el que se usa en el cálculo del coste, el que llegaría si no hubiera un límite de capacidad para cualquier nodo destino.

Ahora, los atributos y funciones de Graph:

Atributos:

- **adjs:** HashMap de pares Integer, Edge. Como solo los sensores se conectan y además lo hacen en una única dirección, no hace falta tener la clásica lista/matriz de adyacencias y así tenemos acceso instantáneo a todas las aristas.
- **Scons/Ccons:** Arrays de enteros con el número de conexiones a cada sensor y centro, se usa para ver la restricción impuesta por el problema sobre estas.
- **Calmacenamiento:** Array con los datos que llegan a cada centro, utilizado para ver si se puede enviar información a un centro.

Funciones:

Primero hay que decir que muchas de estas funciones son auxiliares para los operadores que describiremos más adelante, por ahora basta con saber que se realizan cambios en los elementos a los que apuntan las aristas.

- **simpleSolution/complexSolution:** Ejecutan los algoritmos generadores del estado inicial, añaden aristas a ads, previamente vacío, hasta tener la configuración deseada.

- **enfonsar/desenfonsar:** Propagan respectivamente el efecto de añadir/quitar una arista en cuanto a volumen se refiere. Si añadimos una, intentará enviar datos a su nodo destino y si lo consigue, ese destino a su vez podrá mandar más, siguiendo así hasta llegar a un centro de datos. Quitando una arista es lo mismo pero perdemos volumen de datos.
- **topologicalsort:** Usamos esta función para comprobar si añadir una nueva arista provoca un ciclo en el grafo, lo cual anularía el estado actual, ya que por las restricciones que hemos establecido y las que nos da el problema en nuestro espacio de búsqueda solo tenemos DAGs.
- **switchDestination:** Si las funciones auxiliares determinan que es posible, quitará la arista original, usará a desenfonsar para compensar el borrado y añadirá una nueva arista con el mismo origen y el nuevo destino ya comprobado, por último, hará llamadas a enfonsar para "transmitir" los datos de esta nueva arista.
- **ableSwitch:** Previo a switchDestination, comprueba que el nuevo nodo al que se busca apuntar no sea el que ya era apuntado o el propio origen, además tendrá que cumplir la restricción del límite de conexiones.
- **switchEdges:** Dadas dos aristas, intercambiará los destinos de ambas aplicando la secuencia quitar-desenfonsar-añadir-enfonsar-quitar-desenfonsar-añadir-enfonsar.

2.2. Soluciones iniciales

Para generar las soluciones iniciales nos centramos en mantener los requisitos y criterios de solución de problemas y con el objetivo de minimizar el coste y maximizar el volumen de los datos transmitidos. Implementamos una solución simple y una solución más compleja o voraz.

2.2.1. Solución simple

La solución simple intenta llegar a una solución que maximice el volumen de datos. Para ello, primero conectamos todos los sensores posibles a centros. Luego conectamos el resto de sensores a otros sensores conectados directa o indirectamente a centros. Por supuesto, siempre teniendo en cuenta las restricciones de que no se pueden conectar a un sensor más de tres sensores a la vez y un centro de datos no puede mantener más de 25 conexiones simultáneas.

Tiene una eficiencia de $O(S)$, sea S el número de sensores.

Decidimos usar esta solución simple para que diera buenos resultados y comparar con una solución más compleja. Se justifica por su eficiencia, su maximización del volumen de datos y su cumplimiento de las restricciones del problema.

2.2.2. Solución voraz

La solución más compleja intenta llegar a una solución que casi maximice el volumen de datos y a la vez minimice la distancia. También intenta que sea un buen punto de partida para el algoritmo de Hill Climbing. Para eso, usaremos un set donde poner las aristas posibles a agregar y las ordenaremos por su capacidad y distancia del sensor. Así, se añadirán primero las conexiones de sensores de más capacidad para que puedan almacenar más información de otros sensores. Entonces después de crear el set, añadimos al set las posibles conexiones entre sensores y centros. Luego si un sensor es conectado a un centro, se añadirá al set las conexiones entre los demás sensores hacia ese sensor con la distancia acumulada.

Cuando termina la ejecución, obtenemos una solución con similar volumen de datos a la solución simple pero con la distancia significablemente reducida.

Una cota de la eficiencia de la solución es $O(S^3)$, ya que es la cota de todas las aristas entre sensores. Obviamente generaremos menos aristas, pero seguirá siendo cúbico de forma asintótica.

Decidimos usar esta solución voraz para que se acercara al estado final del algoritmo de Hill Climbing pero no demasiado para que pudiese explorar desde un estado favorable. Se justifica por su cumplimiento de las restricciones y el orden de las aristas por capacidad y distancia acumulada.

2.3. Operadores

Nos costó mucho conseguir operadores que mantuvieran los criterios las restricciones del problema pero al final conseguimos implementar 2. También usamos funciones para calcular el coste y volumen parcialmente al usar cada operador para mejorar la eficiencia general.

2.3.1. Operador de cambio de nodo destino

El primer operador, teniendo en cuenta nuestra representación como DAG, funciona cambiando el nodo destino de una arista por otro nodo destino. Se mantiene la restricción de que todos los sensores tienen que estar conectados directamente o indirectamente a un centro de datos, ya que la solución

inicial nos lo proporciona y cuando conectes el sensor que modificas el nodo destino a otro, este otro seguirá estando conectado al centro.

El único problema es que este operador puede llevar a un ciclo, por lo que comprobamos con un topological sort que no se forme ningún ciclo.

Siendo S el número de sensores y C el número de centros, el factor de ramificación es de $O(S*(S-1+C))$. El coste es el mismo.

El operador se puede aplicar si hay algún sensor o centro que no supere el límite de conexiones y si hay algún nodo con arista. Esto se cumple tanto con las soluciones iniciales como con las diferentes iteraciones de los operadores cuando hay al menos 1 sensor, ya que siempre todos los sensores tendrán 1 arista y siempre habrá algún sensor con 0 conexiones ya que no hay ciclos.

Decidimos incluir este operador ya que tiene un gran factor de ramificación, es eficiente, cumple con las restricciones y llega a soluciones donde el otro operador no puede llegar.

2.3.2. Operador de intercambio de nodos destino

Este segundo operador, por cada 2 sensores, les intercambia sus nodos destino. Se mantienen las restricciones y se comprueba que no se vuelvan a formar ciclos. Siendo S el número de sensores y C el número de centros, el factor de ramificación es de $O(S*(S-1))$. El coste es el mismo.

El operador se puede aplicar si hay 2 nodos con aristas. Esto se cumple tanto con las soluciones iniciales como con las diferentes iteraciones de los operadores cuando hay al menos 2 sensores, ya que siempre todos los sensores tendrán 1 arista.

Decidimos incluir este operador ya que tiene un gran factor de ramificación, es eficiente, cumple con las restricciones y se mueve en la mitad de pasos que el otro operador.

2.4. Heurística

Para la elección de la heurística utilizamos el experimento especial. Primero, comenzamos con una primera versión de la heurística que consistía en restar el volumen de datos al coste. Esto resultaba en el volumen máximo pero no en un valor razonable de distancia.

Al final, escogimos una heurística que devuelve el coste menos el doble del cuadrado de la heurística.

$$\text{coste} - 2 \times \text{volumen}^2$$

Esto es debido a que el coste es una magnitud mucho mayor que el valor de los datos. Con esta ponderación equilibramos aproximadamente ambos

factores.

A parte también la heurística comprueba que nos sigamos moviendo por soluciones que cumplen con las restricciones del problema. Si el estado no cumple las restricciones devuelve el valor máximo de un double en Java.

La eficiencia de nuestra heurística es lineal al número de sensores, ya que tenemos precalculados los valores de volúmenes y distancias y solo los recorreremos dentro de nuestras estructuras.

Nuestra heurística se justifica por el objetivo de equilibrar maximizar el volumen de datos y minimizar el coste de la solución, porque comprueba las restricciones, su eficiencia, y nos da los mejores resultados.

3. Experimentación

Realizaremos 7 experimentos, los primeros de los cuales nos servirán para establecer cuales de los diferentes algoritmos y herramientas implementados utilizaremos. Después analizaremos la eficiencia y calidad de las soluciones con los algoritmos de Hill Climbing y Simulated Annealing, realizando comparaciones entre los dos y explicando lo que hemos observado.

3.1. Elección sobre el conjunto de operadores

Observación

El tipo de operador escogido influye directamente en el factor de ramificación y por consecuencia a la solución que se encuentre.

Planteamiento

Realizaremos ejecuciones usando los 2 operadores disponibles por separado y después usando ambos.

Hipótesis nula

El/los operadores escogidos no influyen en la calidad o el tiempo de la solución.

Método

1. Para todas las ejecuciones usamos la solución inicial voraz con una distribución de 4 centros de datos y 100 sensores.
2. Realizaremos para cada conjunto 10 ejecuciones con semillas distintas.

3. Las ejecuciones usan Hill Climbing y la heurística:

$$h(x) = \text{coste}(x) - 2 \cdot (\text{volumen}(x))^2$$

4. Guardamos los pasos(nodos expandidos), el tiempo y la heurística para cada ejecución con el propósito de comparar la distribución de estos entre los conjuntos de operadores.

Con los datos obtenidos tras las ejecuciones hemos creado las figuras 1 y 2. Tras analizarlas vemos claramente que la hipótesis nula no se cumple, a simple vista se observa una diferencia sustancial entre el conjunto Switch Edges y los otros dos, tanto el tiempo como los pasos siempre serán menores y la calidad de la heurística será peor. Esto se debe a las relaciones heurística-pasos-tiempo; con un conjunto de operadores, dependiendo de la mejora media que se obtenga, Hill Climbing se expandirá más o menos veces, lo ideal es que realice un número de expansiones no demasiado elevado para encontrar una solución óptima, ya que si no el tiempo de ejecución empieza a aumentar.

Entonces, vemos que con el primer conjunto tenemos una heurística horrible porque no conseguimos sucesores aceptables, prácticamente no ascendemos. Con esto podemos fijarnos en los dos conjuntos restantes: Switch Destination y la combinación de ambos Switch. Sabemos que el factor de ramificación del último conjunto es mayor que el de los demás, eso ha hecho que obtengamos una heurística de media ligeramente mejor. Además podemos ignorar la cantidad de pasos, ya que aunque sea mayor, si solo dependiéramos de estos dos valores, preferiríamos una mejor heurística, por muy pequeña que sea la mejora. El problema radica en que para esta ganancia tenemos un tiempo de cómputo notablemente mayor, debido al factor de ramificación. Si bien es cierto que acabamos escogiendo soluciones algo más óptimas, para hacerlo hemos escogido entre una cantidad mucho mayor de sucesores. Al interesarnos un equilibrio hemos decidido únicamente utilizar el operador Switch Destination.

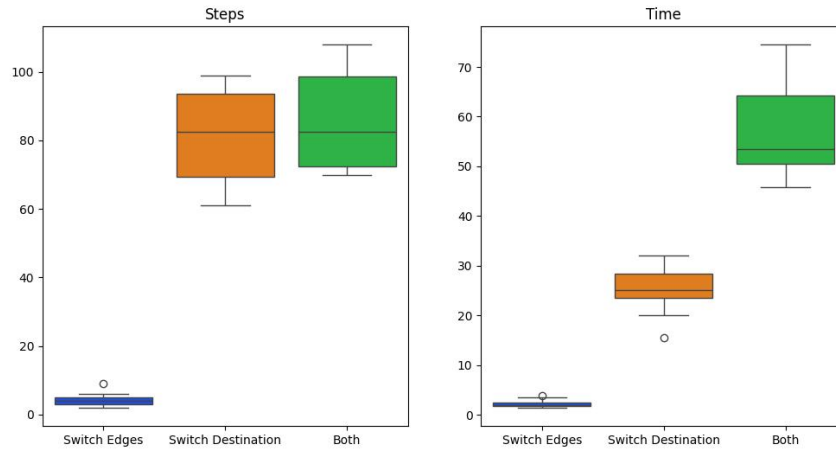


Figura 1: Comparación de los nodos expandidos y el tiempo para cada conjunto de operadores usado

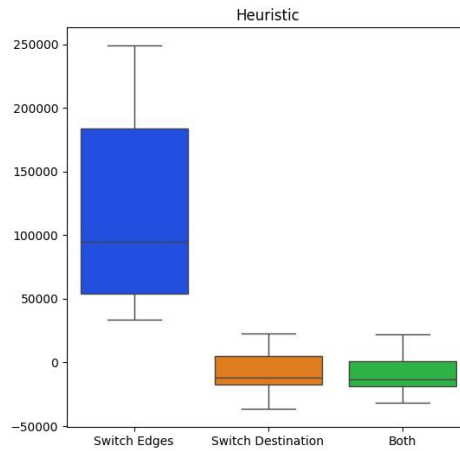


Figura 2: Comparación de las heurísticas

En las siguientes tablas tenemos los resultados en formato numérico. En coincidencia con los gráficos se ve que Switch Edge difiere en todos los aspectos con el resto de conjuntos y entre estos vemos la mejora de la heurística y el empeoramiento notable del tiempo que obtenemos utilizando ambos operadores. Un detalle a tener en cuenta es que las desviaciones no son pequeñas, es decir, la semilla llega a tener cierta influencia sobre cada ejecución aún

teniendo las demás propiedades del algoritmo iguales. Esto se verá también en algunos de los experimentos posteriores.

Operador	Switch Destination	Switch Edges	Ambos
Heurísticas	123258.30	-6182.40	-7289.60
Pasos	4.40	81.60	85.90
Tiempo	2.26	25.23	56.97

Cuadro 1: Medias de resultados para cada conjunto

Operador	Switch Edges	Switch Destination	Both
Heurísticas	79656.81	17935.64	16706.32
Pasos	1.91	12.69	13.45
Tiempo	0.80	4.78	9.15

Cuadro 2: Desviaciones típicas

3.2. Elección de la solución inicial

Observación

Un buen estado inicial podría acercarnos antes a una solución óptima.

Planteamiento

Ejecutaremos Hill Climbing con las 2 soluciones iniciales propuestas.

Hipótesis nula

La solución inicial no afecta al resultado final.

Método

1. Usaremos las mismas heurística y distribución centros/sensores anteriores y el operador escogido en el experimento anterior.
2. Realizaremos 10 ejecuciones con semillas distintas para cada una de las soluciones iniciales.
3. De la misma manera que antes, guardamos los pasos, el tiempo y la heurística medias de cada solución junto a las desviaciones típicas.

Para justificar la elección del algoritmo generador nos basaremos en los mismos 3 parámetros que en el experimento 1 para conseguir la más equilibrada. No obstante, en este caso ha habido una clara mejora con la solución

compleja o voraz respecto a la simple. Se puede ver en los boxplot 3 y 4 que la heurística es algo mejor de media, respecto a los nodos expandidos y el tiempo es mejor aún.

Entonces, empíricamente podemos afirmar que para nuestros problema y algoritmo empezar en un estado con una calidad de solución mejor, por ligera que sea la diferencia, hará que Hill Climbing actúe con más rapidez al estar teóricamente más cerca de un posible óptimo.

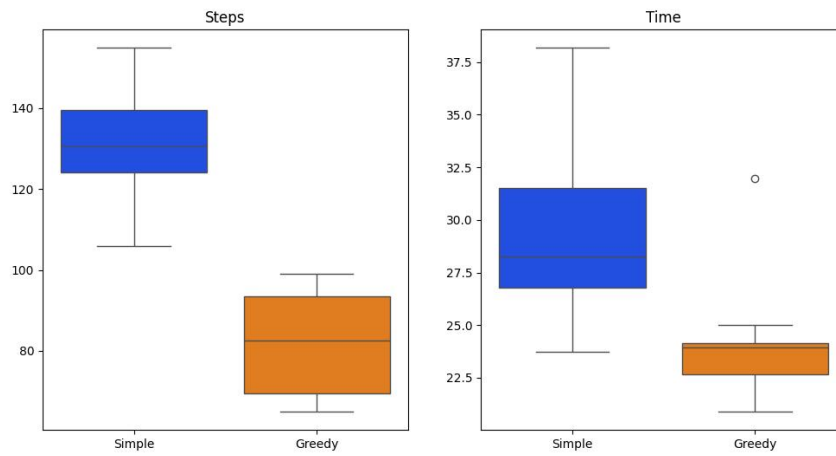


Figura 3: Comparación de los nodos expandidos y el tiempo entre las soluciones iniciales

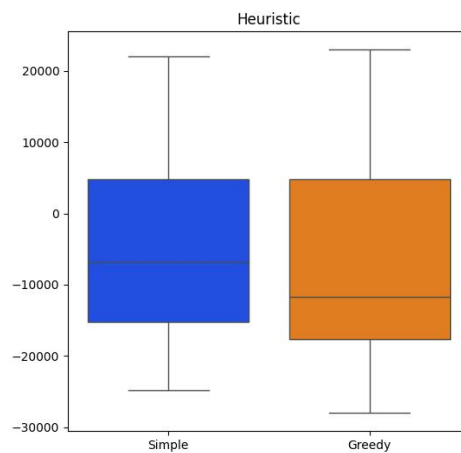


Figura 4: Comparación de las heurísticas

En las tablas se aprecia la mejora total de la solución voraz respecto a la simple en todas las categorías, pero el aspecto a destacar es que si nos fijamos en las columnas de la solución voraz tanto para medias como para desviaciones, veremos que los valores se mantienen similares a los obtenidos en el experimento 1, ya que también usaban esta solución inicial. Reafirma la consistencia de nuestra solución y el operador escogido, las semillas diferentes han tenido un efecto mínimo.

Operador	Simple	Voraz
Heurísticas	-3343.80	-5339.70
Pasos	130.50	82.00
Tiempo	29.65	24.22

Cuadro 3: Medias de resultados para cada solución inicial

Operador	Simple	Voraz
Heurísticas	15443.43	16648.22
Pasos	13.65	12.08
Tiempo	4.83	2.84

Cuadro 4: Desviaciones típicas

3.3. Elección de parámetros para Simulated Annealing

Observación

La técnica de Simulated Annealing puede llevar a estados con mejor heurística que los conseguidos por Hill Climbing.

Planteamiento

Primero, escogemos unos parámetros óptimos para el Simulated Annealing en nuestro problema. Luego, comparamos resultados con el Hill Climbing.

Hipótesis

El Simulated Annealing nos dará mejores resultados de media.

Método

1. Fijaremos el número de iteraciones totales
2. Fijaremos el número de iteraciones por cada cambio de temperatura

3. Fijaremos el número de k y λ y haremos múltiples experimentos.
4. Compararemos la bondad de la solución con el algoritmo de Hill Climbing.

Llamaremos a las variables $niter$, $titer$, k y λ en la explicación, como hace AIMA. Usaremos el escenario inicial de 4 centros de datos y 100 sensores, el operador escogido en el experimento 1 y la solución inicial escogida en el experimento 2 y la función heurística usada en todos los experimentos.

Para la elección de los parámetros, primero empezamos fijando el número total de iteraciones. Para que fuese posible graficar todos los nodos que selecciona el Simulated Annealing o SA, hemos tenido que modificar el código fuente ya que solo devolvía los mejores nodos que encontraba. Con $niter = 10000$, $titer = 10$, $k = 5$ y $\lambda = 0.1$, y una seed random para sensores y centros de datos podemos conseguir esta gráfica.

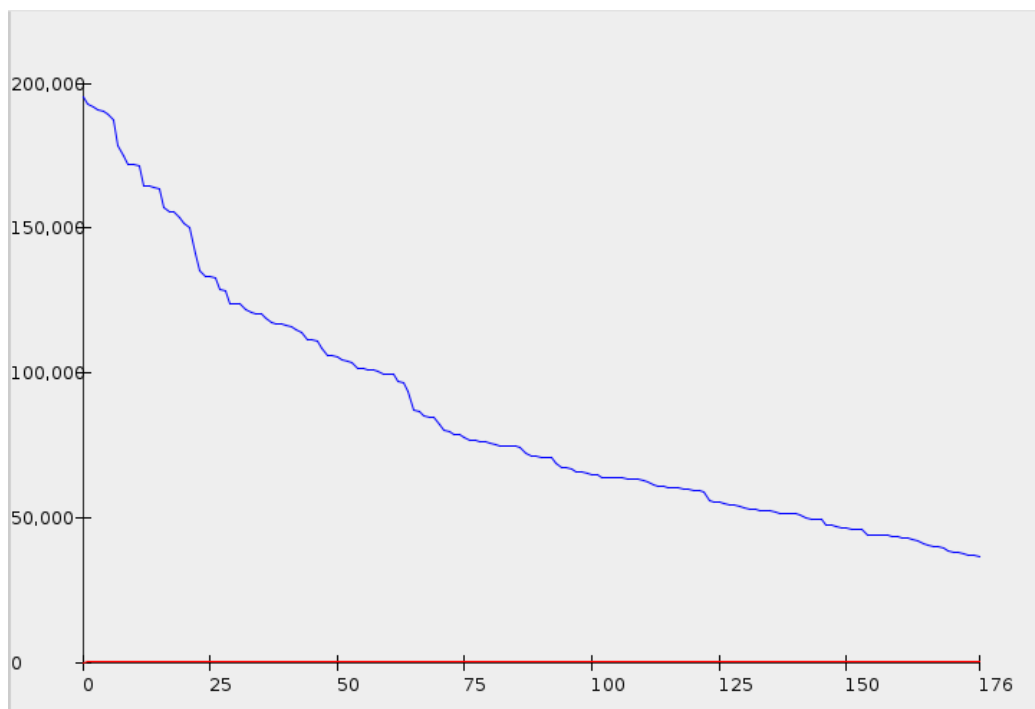


Figura 5: Evolución de la ejecución inicial de SA, el eje X es la heurística del estado y el eje Y son los pasos ejecutados

Podemos observar que el número de iteraciones es suficiente para que converja y lo hemos comprobado para 10 ejecuciones. También podemos apreciar que en ningún momento escoge un nodo que empeora, eso es causa del coste y nuestra heurística que en este escenario van de la magnitud de 500k a

-50k. Ya veremos que escalando la k o la λ podemos llegar a que el SA escoja peores nodos para explorar. Entonces fijaremos el valor de número de iteraciones totales a 10000, ya que llega a converger y no deseamos que empeore la eficiencia a cambio de una ligera mejora de la bondad de la solución. También veremos que en las siguientes ejecuciones, con valores diferentes de titer, λ y k , siguen siendo suficientes para converger.

Para la elección del número de iteraciones por cada cambio de temperatura o titer, decidimos generar 10 escenarios con 2 semillas random y ejecutar cada escenario 10 veces con diferentes valores de titer. Todo esto con los parámetros $niter = 1000$, $k = 5$ y $\lambda = 0.01$.

Valor de titer	1	10	100	1000
Heurísticas	4359.59	3999.59	4101.70	3042.32
Pasos	118.18	117.18	120.13	118.68
Tiempo(ms)	141.57	142.93	148.14	147.99

Cuadro 5: Medias de resultados para cada valor de titer

Observamos en el cuadro 5 que tanto los pasos como el tiempo de ejecución son extremadamente similares. También podemos observar que la mejor heurística es la proporcionada por $titer=10$ aunque también son parecidas teniendo en cuenta la magnitud de la heurística. Fijaremos $titer=10$ para poder seguir fijando el resto de variables.

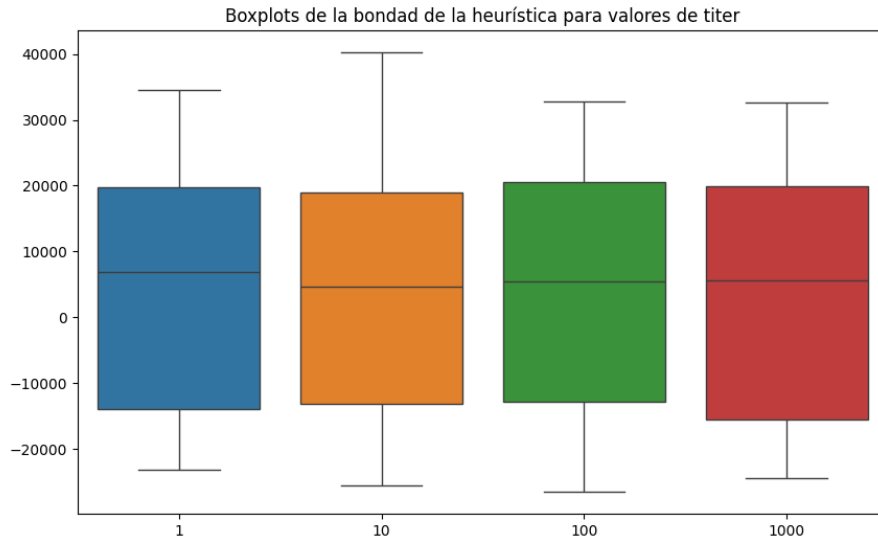


Figura 6: Boxplot de la bondad de la heurística para los diferentes valores de titer

En el boxplot 6 también podemos observar que tanto las medias como desviaciones son muy parecidas pero $\text{titer}=10$ nos da la mejor media y menos desviación.

Ahora empezaremos a intentar encontrar los mejores valores para k y λ . El primer problema que nos podemos encontrar es que en las gráficas con $k=5$ y $\lambda = 0.01$, nunca se escoge un valor que no supera al actual en el SA. Eso se debe al rango de nuestra heurística, ahora veremos unas gráficas del comportamiento variando k en grandes cantidades.

Con $K=5$, veíamos este comportamiento:

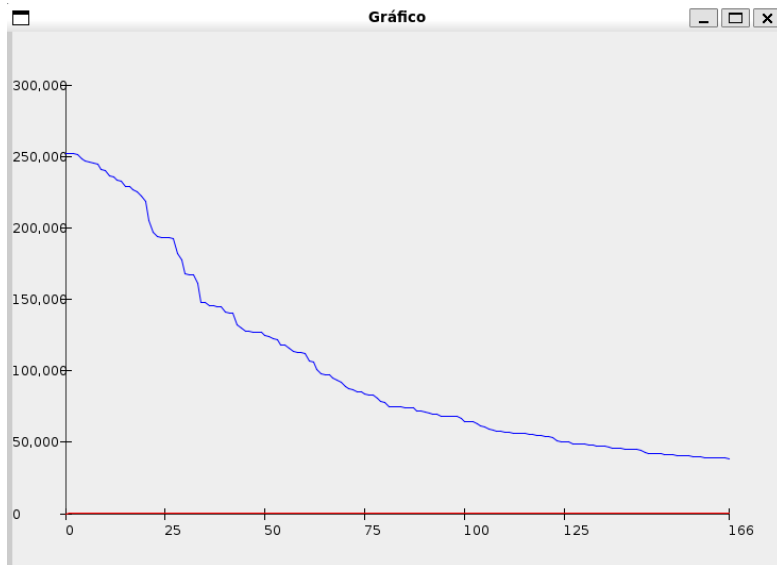


Figura 7: Evolución de la ejecución de SA, el eje X es la heurística del estado y el eje Y son los pasos ejecutados

Para ver que escoge nodos que sean peores hemos tenido que escalar la k a 2000. A partir de esos valores ya acepta peores nodos para la exploración.

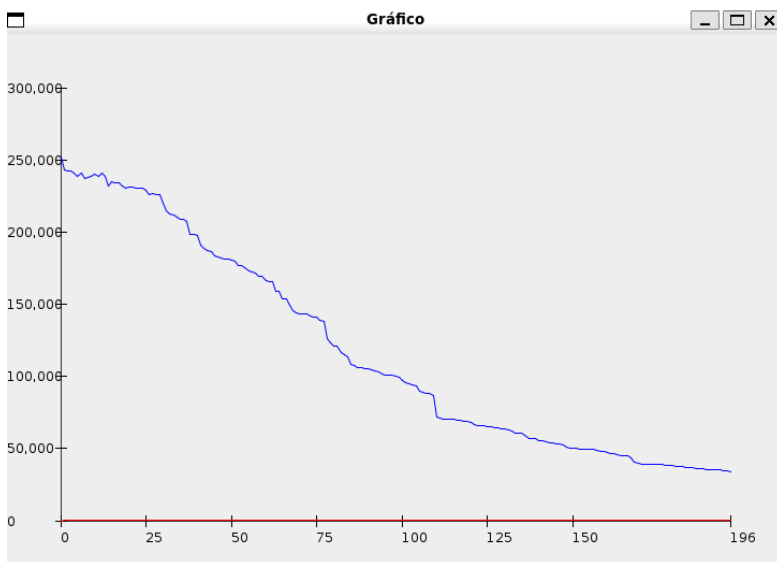


Figura 8: Evolución de la ejecución de SA, el eje X es la heurística del estado y el eje Y son los pasos ejecutados

Si aumentamos la K a más de 2000 o disminuimos la λ el gráfico

muestra muchos más nodos peores aceptados para la exploración. Por ejemplo, con $K=2000$ y $\lambda = 0.001$, obtenemos esta gráfica:

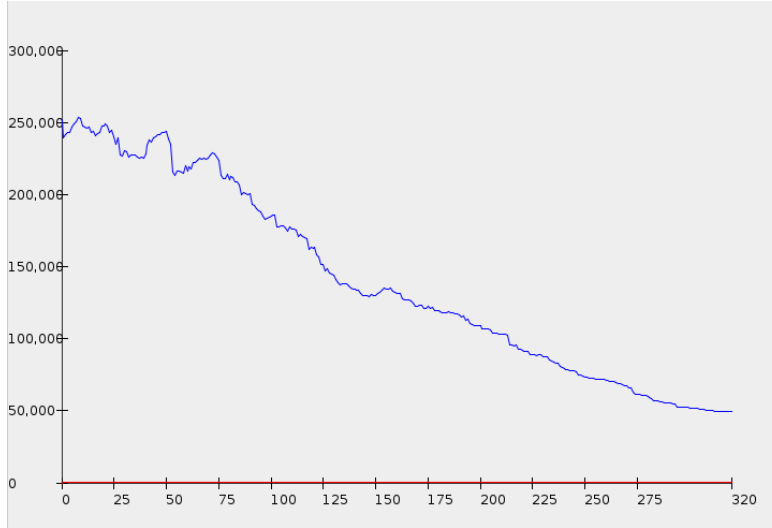


Figura 9: Evolución de la ejecución de SA, el eje X es la heurística del estado y el eje Y son los pasos ejecutados

Para acabar de ajustar la λ y k decidimos hacer varias ejecuciones ajustando sus valores a valores posibles, $\lambda = 0.1, 0.01, 0.001, 0.0001$ y 0.00001 y $k=1, 25, 500, 1000, 2000$. Compararemos los resultados y seleccionaremos al que obtenga mejor media en las 10 ejecuciones de cada 10 escenarios con semillas random compartidas entre las diferentes variables. Generaremos un boxplot a partir de las mejores combinaciones.

K/Lambda	0.1	0.01	0.001	0.0001	0.00001
1	-696.07	-2910.28	-2458.78	-3527.82	-3449.32
5	358.88	-3528.29	-3350.17	-3742.38	-3510.67
25	-1738.75	-2731.21	-3093.64	-3268.25	-3546.21
500	-767.18	-3920.32	-1649.95	6424.07	14959.76
1000	480.35	-2907.93	1314.73	24446.36	39502.02
2000	-433.65	-1439.28	9604.62	52965.05	61122.28

Cuadro 6: Medias de resultados para cada par de variables k y λ

Como podemos observar la λ que causó mejores resultado de media es la 0.01 aunque las 0.0001 y 0.00001 también consiguieron resultados favorables con k pequeñas. Por el lado de las k , vemos que las k menores de 500 y aún más pequeñas dan mejores resultado de media.

Representamos un boxplot de para $\lambda=0.01$:

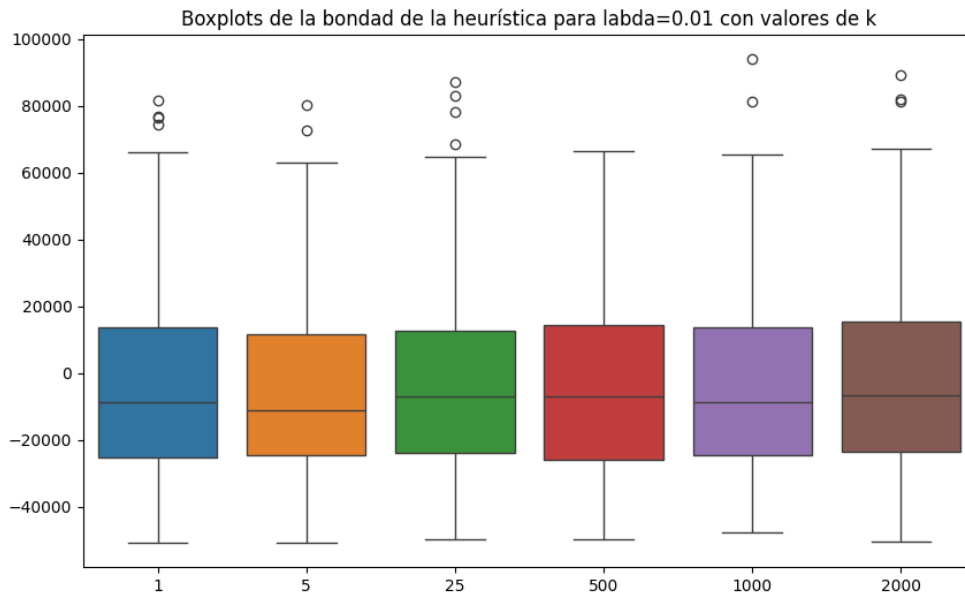


Figura 10

Teniendo en cuenta los resultados empíricos usaremos $k = 500$ y $\lambda = 0.01$.

Por último, compararemos la mejor versión de SA con el algoritmo de Hill Climbing. Escogeremos 10 semillas random para ejecutar los 2 algoritmos en el mismo entorno, y compararemos resultados. Conseguimos estos resultados:

Algoritmo	Hill Climbing	Simulated Annealing
Bondad de la heurística	-18353.0	-7125
Nodos expandidos	79.7	127.5
Tiempo (ms)	6265	158.1

Cuadro 7: Medias para Hill Climbing y Simulated Annealing

Podemos observar como siempre obtenemos una bondad heurística menor con el Hill Climbing de media aunque el SA es mucho más eficiente, se aproxima a sus resultados y selecciona más nodos. Por lo tanto, concluimos que nuestra hipótesis era errónea y el Hill Climbing nos obtiene mejores valores de heurística, aunque el Simulated Annealing sea unas 35 veces más rápido.

3.4. Análisis del tiempo de ejecución

Observación

Es obvio que cuantos más elementos tenga el problema más se tarda en aplicar los algoritmos, queremos ver en qué grado aumenta este tiempo.

Planteamiento

Tenemos que evaluar el crecimiento del tiempo de ejecución con Hill Climbing aumentando el número de centros de 2 en 2 y con los sensores siendo $25 \times$ número de centros.

Hipótesis nula

El tamaño del problema no influye en el tiempo de ejecución.

Método

1. Empezamos con 4 centros y 100 sensores, realizamos 10 ejecuciones con distintas semillas.
2. Aumentamos en 2 los centros y en 50 los sensores, repetimos las veces necesarias y recopilamos los 10 tiempos distintos para cada caso.

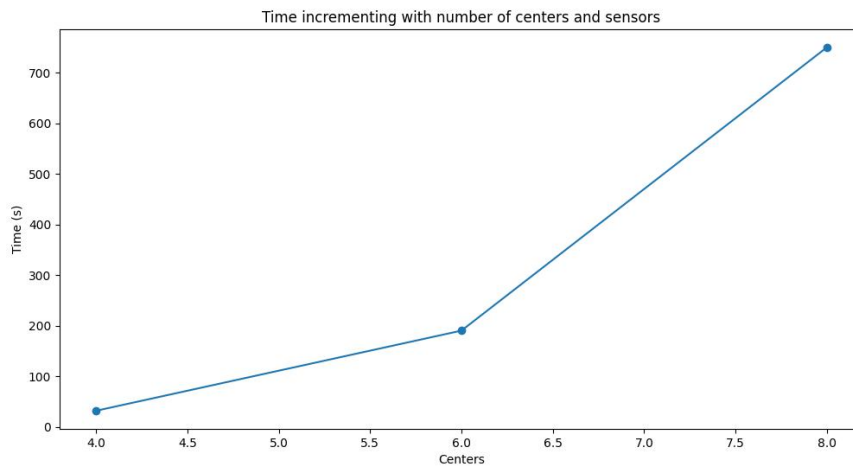


Figura 11: Crecimiento del tiempo de ejecución

Como se ve en el gráfico, solo hay resultados para 4, 6 y 8 centros. Esto se debe a las capacidades de hardware de nuestras máquinas, a partir de los 10 centros una ejecución tardaba aproximadamente una hora. Dicho esto, con

el gráfico se puede observar que el tiempo sigue un crecimiento polinómico de como mínimo segundo grado (ya vemos que no crece de manera lineal).

Se tendría una idea un poco más precisa con más datos pero igualmente se pueden sacar conclusiones. Aumentar los elementos del problema implica también el aumento del factor de ramificación, esto afecta directamente a lo que tardaremos en encontrar la solución óptima.

Núm. Centros	4	6	8
Tiempo (s)	31.72 (3.144)	190.11 (8.62)	749.89 (20.99)

Cuadro 8: Media y desviación para cada caso

3.5. Proporción centros de datos/sensores que acaban en centros vacíos

Observación

Existe una proporción de centros de datos/sensores en la que se encuentran centros con ninguna conexión. Esto sabemos que pasa cuando hay más sensores a centros.

Planteamiento

Haremos varias ejecuciones variando las proporciones para comprobar la hipótesis.

Hipótesis

Hipótesis nula, supondremos que cuando el número de sensores es mayor a los centros no existe un centro con ninguna conexión.

Método

1. Empezamos con 4 centros y 100 sensores y 10 semilla random. Calcularemos la media.
2. Contrastaremos los resultados con una ejecución de 1 centro y otra de 8 centros.
3. Lo contrastaremos también con Simulated Annealing.

Para empezar en nuestros experimentos previos no nos hemos encontrado con ningún centro sin conexiones, ya que eran de proporción 1/25 y veremos

en el experimento que estos efectivamente no resultan en un centro vacío. Siguiendo el método, hacemos la ejecución de 4 centros y 100 sensores, bajando 1 sensor cada iteración. Luego hacemos lo mismo con 1 centro y 8 centros. Los resultados se pueden observar en esta tabla:

Número Centros	1	4	8	16
Número de sensores	0.0	18.9	43.2	76.8

Cuadro 9: Media del números de sensores a partir de los cuales hay un centro sin conexiones para cada número de centros

Se observa que la proporción es muy pequeña, pero cuando aumenta el número de centros de datos, esta proporción disminuye. Son inversamente proporcionales. Las proporciones de los 3 casos son $1/0$, $4/18.9$, $8/43.2$ y $16/76.8$.

En el caso de Simulated Annealing las medias varían ligeramente. Las proporciones de los 3 casos son $1/0$, $4/18.6$, $8/54.3$ y $16/96.2$. Una diferencia que obtenemos en la experimentación es que con el Hill Climbing, siempre que obteníamos el primer número de sensores en el que había un centro sin conexiones para una semilla, si bajábamos el número de sensores siempre seguía habiendo un centro sin conexiones. En cambio, para Simulated Annealing, cuando bajamos el número de sensores a veces aparece un estado final en el que todos los centros tienen alguna conexión. Esto se puede deber al factor de aleatoriedad del Simulated Annealing.

Número Centros	1	4	8	16
Número de sensores	0.0	18.6	54.3	96.2

Cuadro 10: Media del números de sensores a partir de los cuales hay un centro sin conexiones para cada número de centros para SA

Podemos observar los resultados en esta gráfica donde el color amarillo corresponde al algoritmo de Simulated Annealing y el azul corresponde a Hill Climbing.

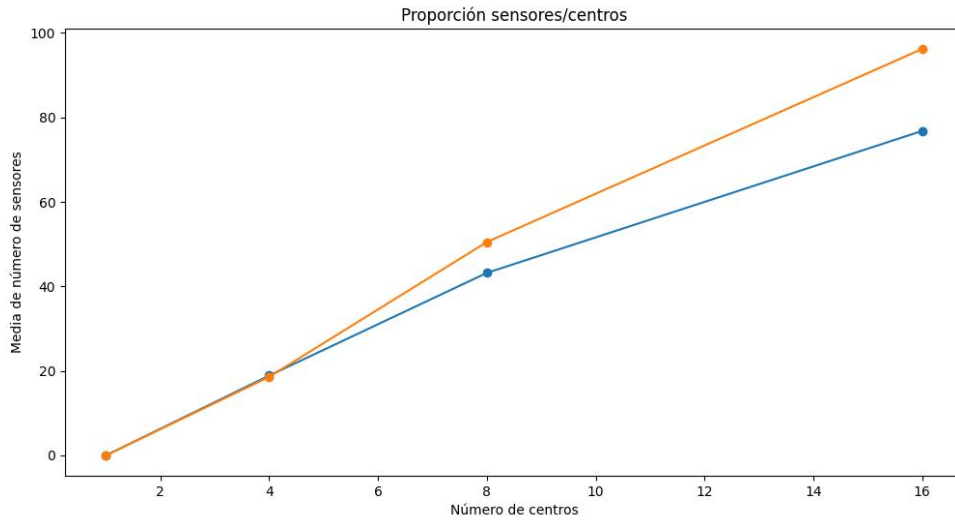


Figura 12: Proporción sensores/centros

Por lo tanto, observamos que nuestra hipótesis era errónea y sí existen casos en los que algún centro no tiene una conexión cuando hay más de un centro. Además de que, cuando aumenta el centro de datos, la proporción disminuye. Esto pasa con tanto Hill Climbing como con Simulated Annealing, aunque el gráfico indica que el ratio es menor con Simulated Annealing.

3.6. Afectación al coste al añadir centros

Observación

Tener más centros pero la misma cantidad de sensores da más opciones de conexión a estos, veremos si mejora o empeora las soluciones.

Planteamiento

Analizamos el efecto por separado para Hill Climbing y SA, manteniendo 100 sensores y empezando con 4 centros.

Hipótesis nula

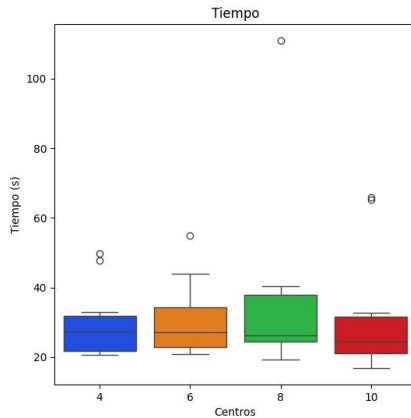
El coste de una solución con n sensores es independiente del número de centros.

Método

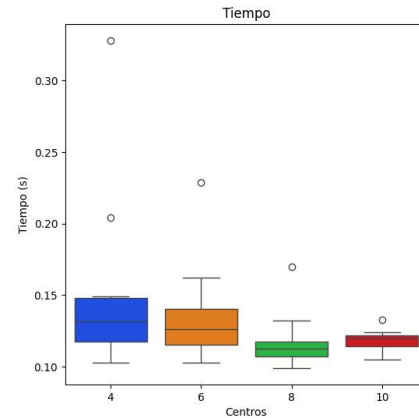
1. Ejecutamos con 10 semillas diferentes y 4 centros para los 2 algoritmos.
2. Aumentamos los centros en 2 y repetimos hasta usar 10.

Aparte de ver el efecto que produce este cambio para cada algoritmo, también queremos comparar ese comportamiento entre los dos. Para empezar, respecto al tiempo y los centros de datos no hay cambios sustanciales. En la figura 13 vemos los tiempos para los dos algoritmos en el dominio de centros usados. En Hill Climbing tenemos un crecimiento más o menos constante y en SA una pequeña mejora al superar los 6 centros, no podemos realizar comparaciones entre los dos algoritmos porque ya se ha visto que actúan en intervalos de tiempo totalmente distintos.

Sobre los centros usados, hemos decidido no graficar los resultados por su insignificancia. De las 20 ejecuciones de Hill Climbing hemos obtenido 3 outliers, ejecuciones en las que no se usaban todos los centros, aunque únicamente había uno sin usar. En SA ha habido un outlier menos y tenían la misma propiedad. Estos resultados concuerdan con los del experimento anterior inmediato, ya que nos afirma que con 100 sensores y 16 centros llenamos todos los centros, sabemos que con menos centros también pasará.



(a) Hill Climbing

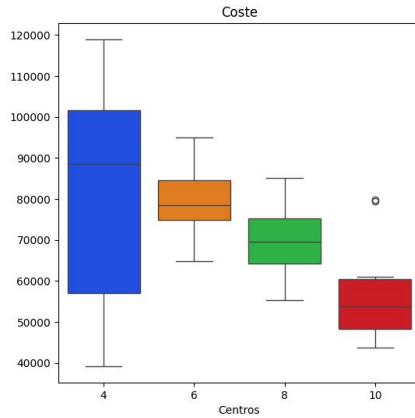


(b) Simulated Annealing

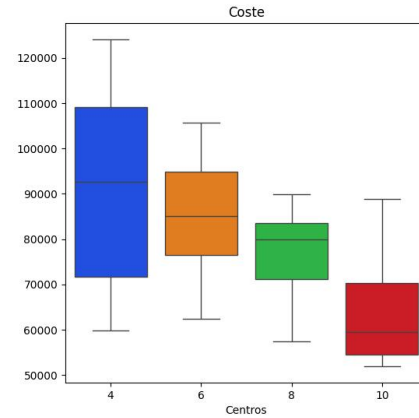
Figura 13: Tiempos

Si bien no hemos observado efectos importantes en los 2 atributos mencionados, el coste es directamente afectado. Es esperable que, con más centros repartidos por el plano, un sensor tenga más opciones de conectarse a un punto que esté más cerca. Hay que recordar que una de las prioridades de nuestra solución voraz es la distancia tanto a los centros como a los sensores ya conectados a centros. En la figura 14 se puede ver que esta propiedad efectivamente nos da soluciones con menor coste a medida que aumenta el

número de centros. Esta evolución es realmente similar entre los 2 algoritmos a pesar de las diferencias que vemos siempre, ya sean la varianza o la media de coste.



(a) Hill Climbing



(b) Simulated Annealing

Figura 14: Costes

En conclusión, este aumento de los centros ofrece una ventaja a nuestro algoritmo para empezar con una solución aún mejor que las habituales y converger a un óptimo con un coste particularmente menor.

3.7. Ajuste de la función heurística para otro escenario

Observación

Ahora trabajaremos en un escenario con 100 sensores y 2 centros donde la función heurística puede afectar al coste y tiempo de ejecución.

Planteamiento

Implementaremos varias heurísticas para cuantificar coste y tiempo de ejecución, con Hill Climbing como pide el enunciado.

Hipótesis

Supondremos que nuestra heurística será la mejor que encontremos para este escenario también.

Método

1. Comparar tiempos y coste con escenario original y heurística original.
2. Comparar entre diferentes heurísticas.

Primero queremos decir que no estamos muy de acuerdo con las afirmaciones del enunciado del experimento. Sí que es verdad que con el escenario original partíamos de nuestra solución original con el máximo de volumen de datos pero con nuestra función heurística llegábamos a un punto donde el coste era cerca del doble de su mínimo y el volumen de datos era la mitad de su máximo. En nuestra heurística le dábamos un peso cuadrático al volumen de datos por lo que sí se tenía en cuenta de la misma forma que el coste.

Empezaremos el experimento ejecutando 100 veces el escenario original y el nuevo estado con 20 semillas compartidas con la heurística original.

Número Centros	2	4
Coste	51073.8	78461.1
Tiempo(s)	7.94	6.44

Cuadro 11: Media del números de sensores a partir de los cuales hay un centro sin conexiones para cada número de centros para SA

Podemos observar que, cuando modificamos el escenario a solo 2 centros, el coste disminuye sustancialmente y el tiempo aumenta sustancialmente.

Para los siguientes pasos de la experimentación modificaremos la ponderización del factor del volumen de datos. La heurística original sigue la fórmula de :

$$\text{coste} - 2 \times \text{volumen}^2$$

Esto se debe a que el coste tiene en cuenta la distancia que está en kilómetros.

Para las siguientes ejecuciones, además de la original, tomaremos estas heurísticas:

$$nada = \text{coste}$$

$$lineal = \text{coste} - \text{volumen}$$

$$cuadratico = \text{coste} - \text{volumen}^2$$

$$cubico = \text{coste} - \text{volumen}^3$$

Después de las ejecuciones obtenemos estos resultados:

Ponderación volumen	0	lineal	cuadrático	original	cúbico
Coste	10030.50	13068.50	13218.60	12440.00	10988.50
Tiempo(s)	27.86	27.87	28.71	27.86	27.86

Cuadro 12: Media del coste y tiempo por cada ponderación del volumen de información

En la tabla ya podemos observar como el tiempo no varía significativamente con la heurística. También se aprecia que el coste crece desde la ponderación = 0 pero a partir de la ponderación cuadrática vuelve a descender.

Concretamente, de 0 a lineal el coste crece una proporción de un 33.02 % respecto al coste de la ponderación = 0. De lineal a cuadrático crece solamente un 1.14 %. De cuadrático a la heurística original descende un 5.86 %. De la heurística original a cúbico descende un 11.67 %.

Esto se observa mejor con este boxplot:

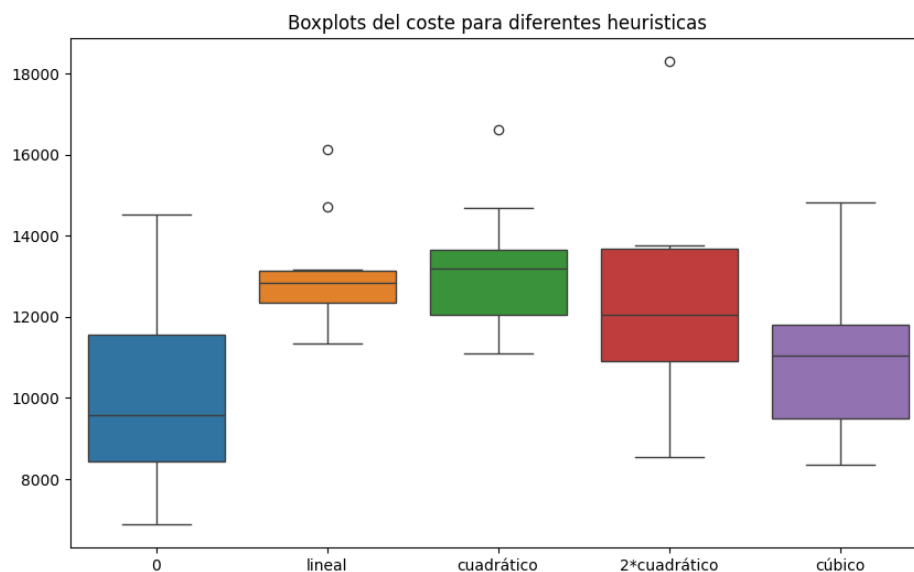


Figura 15: Boxplot del coste para diferentes heurísticas

Aquí se ve claramente como crece el coste en comparación con la ponderación = 0 y luego descende a partir del cuadrático. Sobre todo se ve el descenso entre cuadrático o 2*cuadrático y cúbico.

En el boxplot del tiempo también podemos ver la similitud entre ellos, aunque el cuadrático muestra una desviación estándar mayor.

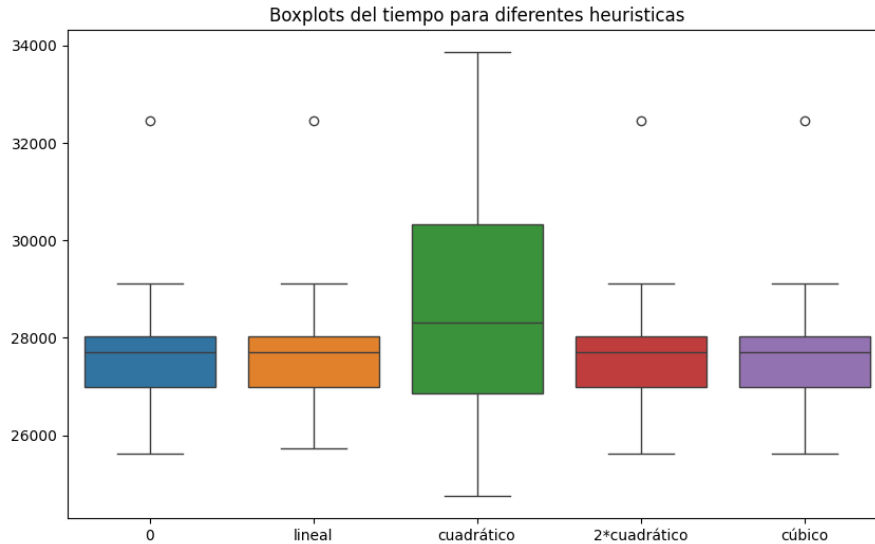


Figura 16: Boxplot del tiempo en ms para diferentes heurísticas

Nuestra hipótesis es errónea ya que hemos visto otras heurísticas, como la de ponderación a 0 y cúbica, que consiguen menor coste que la heurística original.

4. Trabajo de Innovación

Somos David Castro y Aleix Ortigosa, en este pequeño report explicamos el tema escogido y las partes del trabajo que hemos estado realizando hasta hoy.

4.1. Tema

Tras buscar varias compañías con diferentes proyectos, hemos decidido hacer nuestro trabajo sobre la empresa Gray Matter Robotics. La compañía ofrece soluciones robóticas inteligentes y fáciles de implementar para fabricantes con una amplia variedad de productos a través de un formato Robot-as-a-Service (RaaS). Estas soluciones combinan robots industriales, sensores y herramientas de última generación con su software propietario GMR-AI™.

4.2. Reparto del trabajo

Para la realización de este trabajo, hemos dividido las tareas de la siguiente manera:

1. Los dos nos hemos encargado de investigar y redactar sobre las innovaciones de GRM basadas en inteligencia artificial.
2. Aleix ha analizado los beneficios y riesgos de la innovación para la empresa.
3. David ha investigado y escrito sobre los efectos de la innovación en las personas involucradas, detallando los beneficios y riesgos que esta tecnología puede representar para los trabajadores y otros actores relacionados.
4. Ambos trabajaremos a la vez en la creación del documento y la presentación finales, asegurando un informe de calidad que refleje nuestra investigación.

4.3. Referencias

Referencias

- [1] GrayMatter Robotics. "Our Technology." Disponible en: <https://graymatter-robotics.com> (Fecha de consulta: 10 de marzo de 2025).
- [2] LinkedIn. "GrayMatter Robotics Company Profile." Disponible en: <https://www.linkedin.com/company/graymatter-robotics> (Fecha de consulta: 10 de marzo de 2025).
- [3] BuiltIn Los Angeles. "Best Places to Work 2023." Disponible en: <https://builtin.com/los-angeles> (Fecha de consulta: 15 de marzo de 2025).
- [4] Robotics Business Review. "GrayMatter Robotics: AI-Driven Automation for Smart Manufacturing." Disponible en: <https://www.roboticsbusinessreview.com> (Fecha de consulta: 15 de marzo de 2025).
- [5] IEEE Xplore. "Artificial Intelligence in Industrial Robotics: Case Study of GrayMatter Robotics." Disponible en: <https://ieeexplore.ieee.org> (Fecha de consulta: 18 de marzo de 2025).

- [6] TechCrunch. "How AI-Powered Robotics is Transforming Manufacturing." Disponible en: <https://techcrunch.com> (Fecha de consulta: 25 de marzo de 2025).
- [7] Automation World. "GrayMatter Robotics: The Future of AI-Integrated Robots in Industry." Disponible en: <https://www.automationworld.com> (Fecha de consulta: 25 de marzo de 2025).
- [8] The Robot Report. "Innovations in Smart Robotics by GrayMatter Robotics." Disponible en: <https://www.therobotreport.com> (Fecha de consulta: 25 de marzo de 2025).
- [9] Forbes. "How AI is Shaping the Next Generation of Industrial Automation." Disponible en: <https://www.forbes.com> (Fecha de consulta: 25 de marzo de 2025).
- [10] McKinsey & Company. "AI and Robotics in Manufacturing: Case Study on GrayMatter Robotics." Disponible en: <https://www.mckinsey.com> (Fecha de consulta: 25 de marzo de 2025).

4.4. Dificultades

Durante la realización del trabajo, nos hemos enfrentado a las siguientes dificultades:

- **Acceso a información detallada:** Encontrar datos técnicos sobre las innovaciones de la empresa ha sido un reto, ya que la información disponible en línea es general y no siempre detallada en cuanto a las técnicas exactas aplicadas.
- **Análisis de riesgos y beneficios:** Evaluar de manera equilibrada los posibles riesgos y beneficios de la innovación ha requerido suma precisión.
- **Coordinación del trabajo:** Hemos tenido que tener plena comunicación para organizar las tareas y asegurarnos de que ambos aportaremos contenido complementario y sin repeticiones.

Igualmente, creemos que es un tema con realmente pocas dificultades y que nos será fácil hacer una buena presentación.